

MODELO DE RELATÓRIO TÉCNICO

Disciplina: DIM0501 — Boas Práticas de Programação

Tema: Identificação, classificação e priorização de dívida técnica

Semestre: 2026.1

1. Identificação do Grupo

- **Integrante 1:** Rodrigo Ferreira da Nóbrega
- **Integrante 2:** Jorge do Amaral Neto

Linguagem utilizada: Java

Link do repositório Git:

<https://github.com/rodrigofnobrega/boas-praticas-unid2.git>

2. Descrição do Sistema

O sistema é uma aplicação de console, escrita em Java, para o controle de estoque e vendas de uma empresa de varejo. Ele permite cadastrar produtos, registrar vendas com aplicação automática de desconto para compras acima de um valor configurável, listar os produtos cadastrados, gerar um relatório dos itens com estoque baixo e autenticar um usuário administrador.

A aplicação destina-se a pequenos comércios como lojas de roupas, mercearias e livrarias que precisam de um controle simples e direto da entrada e saída de mercadorias. Funcionalidades como o relatório de vendas e a exportação de dados chegaram a ser previstas no código original, mas não estavam implementadas e foram registradas no inventário de dívida técnica.

3. Inventário e Classificação da Dívida Técnica

ID	Localização	Descrição do problema	Tipo de dívida	Esforço	Impacto	Custo
D1	Estoque.java -> variável SENHA_ADMIN. Linha 10	A senha do admin está chumbada no código.	Dívida de configuração.	1	5	5
D2	Import Date. Linha 7	Código não utilizado	Marcadores e código morto	1	1	2
D3	Variáveis produtos e hist. Linhas 12-13	Estruturas de persistência misturadas com lógica de negócio possuindo estado global.	Dívida de arquitetura/design	3	4	3
D4	Variáveis hist. Linha 13	Os valores armazenados não são utilizados.	Dívida de código.	1	2	1
D5	class Produto. Linhas 15	O objeto de Produto está misturado com lógicas de estoque e sem lógica de encapsulamento.	Dívida de arquitetura/design	2	3	3
D6	Linha 21	Comentário desnecessário da função add()	Dívida de documentação	1	1	1

ID	Localização	Descrição do problema	Tipo de dívida	Esforço	Impacto	Custo
D7	método add(). Linhas 22	Nomes ruins nos parâmetros (n, p, q) e nome do método add() dificultam a leitura.	Dívida de código.	1	2	2
D8	método add(). Linhas 27-28	Acoplamento com estado global.	Dívida de arquitetura/design	3	3	3
D9	método add(). Linhas 23,27,28,29	Responsabilidades misturadas	Dívida de arquitetura/design	3	3	3
D10	método vender(). Linha 33	Nomenclatura ruim para índice do for (i).	Dívida de código.	1	2	2
D11	método vender(). Linhas 33-37	Acoplamento com estado global (produtos).	Dívida de arquitetura/design	4	4	4
D12	método vender(). Linhas 33,34,35,39,42,45,50	Responsabilidade excessiva. Busca, valida estoque, calcula de desconto e retorna texto pro usuário.	Dívida de arquitetura/design	3	4	5
D13	método vender(). Linhas 39,40	Números mágicos (100 e 0,1)	Dívida de código	1	3	4

ID	Localização	Descrição do problema	Tipo de dívida	Esforço	Impacto	Custo
D14	método vender(). Linhas 46 e 51.	Tipo de retorno não apropriado. O valor não distingue erro de estoque, produto e total de compra.	Dívida de robustez	3	4	5
D15	método calcular_total(). Linha 55	O nome do método não segue padrão CamelCase.	Dívida de código	1	1	2
D16	método calcular_total(). Linhas 57,58	Lógica de desconto conflitante com a existente no método de vender()	Dívida de código	2	5	5
D17	método calcular_total(). Linha 55	Função não está sendo utilizada em nenhum lugar. Vai precisar implementar o código dela.	Marcados e código morto	5	1	1
D18	listar()	O nome não explicita sobre o que será listado. Depende do estado global.	Dívida de código e Dívida de arquitetura/design	1	2	2
D19	relatorio_estoque_baixo(). Linha 71	O nome do método não segue o padrão	Dívida de código	1	1	2

ID	Localização	Descrição do problema	Tipo de dívida	Esforço	Impacto	Custo
		CamelCase. Número mágico. Responsabilidade excessiva. Depende do estado global.	e Dívida de arquitetura/design			
D20	Linha 81-90	Código Morto	Marcadores e código morto:	1	1	1
D21	método relatorio_vendas(). Linha 92	Nomenclatura não segue o padrão CamelCase e código não implementado	Dívida de Código. Marcadores e código morto	1	1	2
D22	main(). Linhas 102, 110, 117, 118, 120, 128	Número mágico na lógica dos IFs, poderia ser variável ou enum.	Dívida de código.	2	3	3
D23	main() Linha 97	Acoplamento com o Scanner.	Dívida de arquitetura/design:	2	2	2
D24	main(). Linha 104, 106, 108, 111, 113, 122	Falta de validação de entradas nos inputs. Quebrando se o tipo for errado ou nome composto	Dívida de robustez.	3	4	3

ID	Localização	Descrição do problema	Tipo de dívida	Esforço	Impacto	Custo
D25	main(). Linhas 121 - 127	Realiza a validação se o usuário é um admin.	Dívida de arquitetura/design	2	5	5
D26	main(). Código todo	Responsabilidade Excessiva. Lê entradas de usuário, redireciona para métodos, printa menu, loga admin.	Dívida de arquitetura/design	4	4	5
D27	Código todo	Falta de documentação apropriada DocStrings	Dívida de documentação.	3	4	5
D28	Código todo	Falta de testes de código	Dívida de testes.	5	5	5

4. Matriz de Priorização

Considerando o esforço de 1-2 como baixo e 3-5 como alto, o impacto de 1-2 como baixo e 3-5 como alto e utilizando o critério a seguir para calcular a prioridade: $\text{prioridade} = (\text{impacto} + \text{custo}) / \text{esforço}$. Obteremos a matriz de priorização abaixo:

	Esforço baixo	Esforço alto
Impacto alto	D1, D13, D16, D25, D5, D22	D12, D14, D27, D3, D24, D26, D8, D9, D11, D28
Impacto baixo	D7, D10, D18, D2, D4, D15, D19, D21, D23, D6, D20	D17

5. Roadmap de Quitação

Pagar agora (quitados neste trabalho)

Itens de alto impacto e baixo esforço.

- D1 - Senha do administrador chumbada no código: externalizada para um arquivo .env.
- D5 - Classe Produto sem encapsulamento: campos tornados privados, com construtor e métodos de acesso.
- D13 - Números mágicos (100 e 0,1) em vender(): substituídos por constantes nomeadas.
- D16 - Regra de desconto divergente entre vender() e calcular_total(): unificada em um único método.
- D22 - Números mágicos nas opções do menu do main(): substituídos por constantes nomeadas.
- D25 - Autenticação de administrador embutida no main(): extraída para o método autenticarAdmin().

Pagar depois

Itens importantes, porém de alto esforço, que dependem de refatoração arquitetural ou de infraestrutura de testes:

- D3, D8, D9, D11, D12 e D26 - separação de responsabilidades e eliminado do estado global estático para camadas de apresentação, serviço e persistência.
- D14 - tipo de retorno de vender() que não distingue o valor retornado do erro para o valor retornado do total da compra.
- D17 - implementação de fato do relatório que consumiria cálculo total().
- D24 - validação de entradas no main() (atualmente quebra com tipo errado).
- D27 - documentação apropriada em todo o código.
- D28 - suite de testes automatizados.

Aceitar conscientemente

Itens de baixo impacto cujo custo de correcao nao compensa agora:

- D2 e D20 - código morto (import Date e método exportar() comentado), mantidos por ora por terem impacto desprezível.
- D4 - variável global hist sem uso relevante.
- D6, D7, D10, D15, D18, D19, D21 e D23 - ajustes de nomenclatura (CamelCase, nomes de parâmetros e de índices de laço), comentários redundantes e acoplamento ao Scanner.

6. Itens Quitados (Antes vs Depois)

Item 1 - (ID: D1 - Senha do administrador chumbada no codigo)

Antes:

```
static String SENHA_ADMIN = "1234"; // senha do admin
```

Depois:

```
private static final String SENHA_ADMIN = carregarSenhaAdmin();
```

```
private static String carregarSenhaAdmin() {  
    final String chave = "ESTOQUE_ADMIN_SENHA";  
    try (BufferedReader br = new BufferedReader(new  
FileReader(".env"))) {  
        // ... le linhas no formato CHAVE=valor e retorna o valor  
    }  
}
```



```

    } catch (IOException e) { /* .env ausente: usa fallbacks */ }
    String doAmbiente = System.getenv(chave);
    if (doAmbiente != null && !doAmbiente.isEmpty()) return
doAmbiente;
    return "admin-dev";
}
Arquivo .env:
# .env
ESTOQUE_ADMIN_SENHA=1234

```

Explicação: a senha deixou de ficar chumbada no código versionado e passou a ser lida de um arquivo .env. A constante tornou-se private static final, e o método carregarSenhaAdmin() ainda oferece fallback para variável de ambiente do sistema e, por último, um valor padrão de desenvolvimento. Comportamento preservado: com a senha definida no .env, a autenticação continua funcionando normalmente.

Item 2 - (ID: D5 - Classe Produto sem encapsulamento)

Antes:

```

static class Produto {
    String nome;
    double preco;
    int qtd;
}

```

Depois:

```

static class Produto {
    private String nome;
    private double preco;
    private int qtd;
}

```

```

    Produto(String nome, double preco, int qtd) {
        this.nome = nome; this.preco = preco; this.qtd = qtd;
    }

```

```

    String getNome() { return nome; }
    double getPreco() { return preco; }
    int getQtd() { return qtd; }
    void setQtd(int qtd) { this.qtd = qtd; }
}

```

Explicação: os campos públicos, antes manipulados diretamente por toda a classe (ex.: produtos.get(i).qtd = ...), foram encapsulados como private e passaram a ser acessados por construtores e métodos de acesso. Os métodos add(), vender(), listar() e relatorio_estoque_baixo() foram atualizados para usar os métodos de acesso.

Item 3 - (ID: D13 - Números mágicos na regra de desconto)

Antes:

```
if (total > 100) {                // limite magico
    total = total - total * 0.1;  // percentual magico
}
```

Depois:

```
private static final double LIMITE_PARA_DESCONTO = 100.0;
private static final double PERCENTUAL_DESCONTO = 0.10;
...
if (total > LIMITE_PARA_DESCONTO) {
    return total - total * PERCENTUAL_DESCONTO;
}
```

Explicação: os valores 100 e 0,1, antes embutidos diretamente em vender(), foram promovidos a constantes nomeadas (LIMITE_PARA_DESCONTO e PERCENTUAL_DESCONTO). O significado de cada número fica explícito e qualquer ajuste de política de desconto passa a ser feito em um único lugar.

Item 4 - (ID: D16 - Regra de desconto divergente entre metodos)

Antes:

```
// em vender():
if (total > 100) { total = total - total * 0.1; }
```

```
// em calcular_total():
if (t > 200) { t = t - t * 0.15; } // limite e desconto
diferentes!
```

Depois:

```
static double aplicarDesconto(double total) {
    if (total > LIMITE_PARA_DESCONTO) {
        return total - total * PERCENTUAL_DESCONTO;
    }
    return total;
}
```

```
// vender():          total = aplicarDesconto(preco * quantidade);  
// calcular_total(): return aplicarDesconto(preco * quantidade);
```

Explicação: existiam duas regras de desconto divergentes, 10% acima de R\$100 em vender() e 15% acima de R\$200 em calcular_total(), o que produzia totais inconsistentes conforme o caminho do código. As regras foram unificadas no método aplicarDesconto().

Item 5 - (ID: D22 - Números mágicos nas opções do menu)

Antes:

```
if (op.equals("1")) { ... }  
else if (op.equals("2")) { ... }  
...  
else if (op.equals("0")) { break; }
```

Depois:

```
private static final String OPCA0_CADASTRAR      = "1";  
private static final String OPCA0_VENDER        = "2";  
private static final String OPCA0_LISTAR        = "3";  
private static final String OPCA0_ESTOQUE_BAIXO = "4";  
private static final String OPCA0_ADMIN         = "5";  
private static final String OPCA0_SAIR          = "0";  
...  
if (op.equals(OPCA0_CADASTRAR)) { ... }  
else if (op.equals(OPCA0_VENDER)) { ... }  
...  
else if (op.equals(OPCA0_SAIR)) { break; }
```

Explicação: as strings "1", "2", ... "0" espalhadas pelos IFs do método main() foram substituídas por constantes nomeadas, deixando claro o que cada opção significa e reduzindo o risco de erros ao comparar as opções.

Item 6 - (ID: D25 - Autenticação de admin embutida no main())

Antes:

```
System.out.print("Senha: ");
String s = sc.next();
if (s.equals(SENHA_ADMIN)) {
    System.out.println("Acesso liberado");
} else {
    System.out.println("Senha errada");
}
```

Depois:

```
static boolean autenticarAdmin(String senha) {
    return SENHA_ADMIN.equals(senha);
}
```

```
// no main():
if (autenticarAdmin(s)) {
    System.out.println("Acesso liberado");
} else {
    System.out.println("Senha errada");
}
```

Explicação: a regra que autentica o administrador estava embutida no fluxo de menu do main(). Ela foi extraída para o método autenticarAdmin(), separando a lógica de autenticação da interface de console. Isso reduz a responsabilidade do main() e deixa a autenticação reutilizável e mais fácil de evoluir no futuro.

7. Conclusão

A dívida técnica não se resume a apenas código feio, ela é um passivo que cresce com atalhos tomados sob pressão de prazo e que cobra juros na forma de bugs, retrabalho e dificuldade de evolução. Foi identificado diversos tipos distintos de dívida como: arquitetura, código, robustez, configuração, documentação, testes e código morto, e que cada uma exige um tratamento diferente.

A maior dificuldade consistiu na atribuição das notas de impacto, esforço e custo, uma vez que essa avaliação exige muita subjetividade, além disso, fórmula $\text{prioridade} = (\text{impacto} + \text{custo}) / \text{esforço}$ ajudou a ordenar quais tarefas devem ser realizadas primeiro.

A visão do grupo sobre qualidade de código melhorou, pois foi possível identificar vários problemas diferentes em um mesmo código e também por ter sido realizado a refatoração de alguns problemas, melhorando análises futuras de código.

Portanto, fica claro que a gestão de dívidas técnicas é essencial para garantir a qualidade e a sustentabilidade do software a longo prazo, onde as dívidas identificadas são quitadas utilizando critérios claros de classificação e prioridade.